VIT-AP CSE4006 – DEEP LEARNING

FAT FULLY SOLVED 10/10 ANSWERS

WINTER 2025-26

Q1(a) Scale Down Matrix A to 0–1 Range Using Matrix B and Hadamard Product

Given Matrix

$$A = \begin{bmatrix} 12 & 45 & 200 & 123 & 67 & 34 & 90 & 150 & 210 & 45 & 255 & 180 \\ 75 & 30 & 90 & 80 & 100 & 120 & 140 & 160 & 200 & 220 \end{bmatrix}$$

Step 1: Convert Matrix A into 0–1 Range

To scale image pixel values into 0–1 range:

$$B = \frac{A}{255}$$

Matrix B

$$B \approx \begin{bmatrix} 0.047 & 0.176 & 0.784 & 0.482 & 0.263 & 0.133 & 0.353 & 0.588 & 0.824 & 0.176 & 1.000 & 0.706 \\ 0.294 & 0.118 & 0.353 & 0.314 & 0.392 & 0.471 & 0.549 & 0.627 & 0.784 & 0.863 \end{bmatrix}$$

Step 2: Hadamard Product

Hadamard Product means element-wise multiplication.

$$C = A \odot B$$

Each element is multiplied with its corresponding element.

Example:

$$12 \times 0.047 = 0.564$$

Final Hadamard Product Result

$$C \approx \begin{bmatrix} 0.564 & 7.92 & 156.8 & 59.286 & 17.621 & 4.522 & 31.77 & 88.2 & 173.04 & 7.92 & 255 & 127.08 & 22.05 & 3.54 & 31.77 & 2 \end{bmatrix}$$

Conclusion

Scaling converts all values into a standard range which improves:

- Neural network convergence
- Numerical stability
- Faster training
- Better optimization

Hadamard product is widely used in:

- CNN operations
- Attention mechanisms
- LSTM gates
- Feature masking

==========

Q1(b) Check Whether Matrix is Orthogonal and Find Inverse

Given Matrix

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{bmatrix}$$

Definition of Orthogonal Matrix

A matrix is orthogonal if:

$$A^T A = I$$

Also:

$$A^{-1} = A^T$$

Step 1: Find Transpose

$$A^T = \begin{bmatrix} 1 & 0 & 5 \\ 2 & 1 & 6 \\ 3 & 4 & 0 \end{bmatrix}$$

Step 2: Compute AᵀA

$$A^T A \;/\; I \quad =$$

Therefore:

Final Conclusion

The given matrix is NOT orthogonal.

Step 3: Find Determinant

$$|A| = 1(1 \times 0 - 4 \times 6) - 2(0 \times 0 - 4 \times 5) + 3(0 \times 6 - 1 \times 5)$$

$$= 1(0 - 24) - 2(0 - 20) + 3(0 - 5)$$

$$= -24 + 40 - 15$$

$$= 1$$

Step 4: Inverse Matrix

$$A^{-1} = \begin{bmatrix} -24 & 18 & 5 \\ 20 & -15 & -4 \\ -5 & 4 & 1 \end{bmatrix}$$

Conclusion

Since:

$$A^T A \;/\; I \quad =$$

The matrix is NOT orthogonal.

However inverse exists because determinant is non-zero.

==========

Q2 Meteorological Prediction Problem

A meteorological department is analyzing rainfall patterns.

Given features:

- Time
- Temperature
- Humidity
- Rainfall
- Rain occurrence

We need:

1. Linear regression model for rainfall prediction
2. Logistic regression for rain classification

==========

Part A – Linear Regression

Linear Regression Definition

Linear regression predicts continuous values.

General equation:

$$y = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Where:

- x_1 = Time
- x_2 = Temperature
- x_3 = Humidity
- y = Rainfall

Model Equation

$$Rainfall = w_1(Time) + w_2(Temperature) + w_3(Humidity) + b$$

Interpretation

- Higher humidity increases rainfall probability.
- Lower temperature often supports rainfall.
- Rainfall pattern depends on atmospheric conditions.

==========

Part B – Logistic Regression

Logistic Regression Definition

Used for binary classification.

Output:

- 1 → Rain
- 0 → No Rain

Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where:

$$z = w_1 x_1 + w_2 x_2 + b$$

Prediction for Temp = 29°C and Humidity = 75%

Observing training data:

- Humidity above 70 mostly corresponds to rain.
- Temperature around 29 with high humidity indicates rainfall.

Therefore:

$$P(Rain) > 0.5$$

Final Answer

Prediction:

Rain is likely to occur.

==========

Q3 Backpropagation Using Batch Gradient Descent

Given

- Error Function:

$$E = \frac{1}{2}\sum(\hat{y} - y)^2$$

- Learning rate:

$$\eta = 0.3$$

Definition of Backpropagation

Backpropagation is a supervised learning algorithm used to train neural networks.

It updates weights by propagating error backward from output layer to input layer.

==========

Steps in Backpropagation

Step 1: Initialize Weights

Weights are initialized randomly.

Step 2: Forward Propagation

Compute:

$$z = wx + b$$

Apply activation function.

Generate prediction:

$$\hat{y}$$

Step 3: Compute Error

$$E = \frac{1}{2}(\hat{y} - y)^2$$

Step 4: Compute Gradients

Using chain rule:

$$\frac{\partial E}{\partial w}$$

Step 5: Update Weights

$$w_{new} = w_{old} - \eta \frac{\partial E}{\partial w}$$

Step 6: Repeat Until Convergence

Training continues until loss becomes minimum.

==========

Advantages

- Efficient training algorithm
- Works for deep neural networks
- Minimizes prediction error
- Learns complex nonlinear patterns

==========

Challenges

- Vanishing gradients
- Exploding gradients
- Slow convergence
- Computational complexity

==========

Conclusion

Backpropagation combined with Batch Gradient Descent allows neural networks to learn optimal weights by minimizing error iteratively.

==========

Q4 Adaptive Moment Estimation (Adam Optimizer)

Definition

Adam optimizer combines:

- Momentum
- RMSProp

It computes adaptive learning rates for every parameter.

==========

Step 1: First Moment Estimate

Computes moving average of gradients.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Where:

- m_t = first moment
- g_t = gradient
- β_1=0.9 typically

Step 2: Second Moment Estimate

Computes moving average of squared gradients.

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Where:

- v_t=second moment
- β_2=0.999 typically

Step 3: Bias Correction

Corrects initialization bias.

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Step 4: Parameter Update Rule

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Where:

- α=learning rate
- ϵ=small constant

==========

Advantages of Adam

- Fast convergence
- Adaptive learning rates
- Works well for sparse gradients
- Requires less tuning
- Widely used in deep learning

==========

Applications

- CNN
- RNN
- Transformers
- GANs
- NLP models

==========

Conclusion

Adam is one of the most efficient optimizers because it combines momentum and adaptive learning rate mechanisms.

==========

Q5(a) Inception Block in GoogLeNet

Definition

Inception block is a CNN architecture module that performs multiple convolutions in parallel.

It captures multi-scale features efficiently.

==========

Parallel Paths in Inception Block

1. 1×1 Convolution
2. 1×1 → 3×3 Convolution
3. 1×1 → 5×5 Convolution
4. Max Pooling → 1×1 Convolution

==========

Role of 1×1 Convolution

1×1 convolutions are used for:

- Dimensionality reduction
- Reducing computational cost

- Increasing efficiency

==========

Advantages of Inception Block

- Multi-scale feature extraction
- Lower computational complexity
- Better accuracy
- Efficient architecture
- Reduced overfitting

==========

Applications

- Image classification
- Object detection
- Medical imaging
- Face recognition

==========

Conclusion

GoogLeNet uses inception blocks to efficiently learn both local and global features.

==========

Q5(b) Inception Block Numerical

Given

Input feature map:

$$28 \times 28 \times 192$$

Configuration:

- 1×1 conv → 64 filters
- 1×1→3×3 : 96→128 filters
- 1×1→5×5 : 16→32 filters
- Max pooling→1×1 : 32 filters

==========

Step 1: Output Depth After Concatenation

Total channels:

$$64 + 128 + 32 + 32$$

$$= 256$$

Final Output Depth

$$256$$

==========

Step 2: Output Size of Each Branch

Padding preserves dimensions.

Therefore each branch output:

$$28 \times 28$$

After concatenation:

$$28 \times 28 \times 256$$

==========

Step 3: Parameter Calculation

Branch 1 – 1×1 Conv

$$(1 \times 1 \times 192 + 1) \times 64$$

$$= (192 + 1) \times 64$$

$$= 12352$$

Branch 2 – 1×1 Reduction

$$(1 \times 1 \times 192 + 1) \times 96$$

$$= 18528$$

Branch 2 – 3×3 Conv

$$(3 \times 3 \times 96 + 1) \times 128$$

$$= (864 + 1) \times 128$$

$$= 110720$$

Branch 3 – 1×1 Reduction

$$(1 \times 1 \times 192 + 1) \times 16$$

$$= 3088$$

Branch 3 – 5×5 Conv

$$(5 \times 5 \times 16 + 1) \times 32$$

$$= (400 + 1) \times 32$$

$$= 12832$$

Branch 4 – Pool Projection

$$(1 \times 1 \times 192 + 1) \times 32$$

$$= 6176$$

==========

Total Parameters

$$12352 + 18528 + 110720 + 3088 + 12832 + 6176$$

$$= 163696$$

==========

Final Answer

Output feature map:

$$28 \times 28 \times 256$$

Total learnable parameters:

$$163696$$

==========

Q6 Convolution + Leaky ReLU + Max Pooling

Given Input Matrix

$$\begin{bmatrix} 2 & 4 & 6 & 8 & 10 \\ 1 & 3 & 5 & 7 & 9 \\ 0 & 2 & 4 & 6 & 8 \\ 1 & 1 & 2 & 2 & 3 \\ 5 & 4 & 3 & 2 & 1 \end{bmatrix}$$

==========

Filter

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Stride = 1

Padding = valid

==========

Step 1: Output Size

$$\begin{aligned} Output &= \frac{N - F + 2P}{S} + 1 \\ &= \frac{5 - 3 + 0}{1} + 1 \\ &= 3 \end{aligned}$$

Therefore output feature map size:

$$3 \times 3$$

==========

Step 2: Convolution Operation

First Element

$$(2 \times 1) + (4 \times 0) + (6 \times -1)$$
$$+(1 \times 1) + (3 \times 0) + (5 \times -1)$$
$$+(0 \times 1) + (2 \times 0) + (4 \times -1)$$
$$= 2 + 0 - 6 + 1 + 0 - 5 + 0 + 0 - 4$$
$$= -12$$

Similarly compute all positions.

==========

Convolution Output

$$\begin{bmatrix} -12 & -12 & -12 & -9 & -9 & -9 & 0 & 0 & 0 \end{bmatrix}$$

==========

Step 3: Apply Leaky ReLU

Formula:

$$f(x) = x, x > 0$$
$$f(x) = \alpha x, x < 0$$

Given:

$$\alpha = 0.01$$

==========

After Leaky ReLU

$$\begin{bmatrix} -0.12 & -0.12 & -0.12 & -0.09 & -0.09 & -0.09 & 0 & 0 & 0 \end{bmatrix}$$

==========

Step 4: Max Pooling

Pool size:

$$2 \times 2$$

Stride:

$$1$$

Take maximum from each region.

==========

Final Max Pooled Output

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

==========

Conclusion

- Convolution extracts spatial features.
- Leaky ReLU introduces nonlinearity.
- Max pooling reduces dimensions and computation.

==========

Q7 Backpropagation Through Time (BPTT)

Definition

BPTT is the training algorithm used for Recurrent Neural Networks.

It extends standard backpropagation across time steps.

==========

Working of BPTT

Step 1: Unroll RNN Through Time

RNN is expanded across time steps.

Step 2: Forward Propagation

Compute hidden states and outputs sequentially.

Step 3: Compute Total Loss

Loss is summed over all time steps.

Step 4: Backward Propagation Through Time

Gradients are propagated backward across all previous states.

Step 5: Weight Update

Weights updated using gradient descent.

==========

Challenges in BPTT

1. Vanishing Gradient

Gradients become very small.

Effects:

- Poor long-term memory
- Slow learning

---

2. Exploding Gradient

Gradients become extremely large.

Effects:

- Numerical instability
- Divergence

---

3. Computational Complexity

Long sequences require:

- Large memory
- High computation time

==========

Solutions

- LSTM
- GRU
- Gradient clipping
- Attention mechanisms

==========

Conclusion

BPTT enables RNNs to learn sequential dependencies but suffers from gradient-related issues for long sequences.

==========

Q8 LSTM Numerical Problem

Given

Input vector:

$$x_t = [0.6, -0.1]$$

Previous hidden state:

$$h_{t-1} = [0.2, 0.5]$$

Previous cell state:

$$C_{t-1} = [0.4, 0.1]$$

==========

LSTM Equations

Forget Gate

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Input Gate

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Candidate Cell State

$$\tilde{C}_t = tanh(W_c[h_{t-1}, x_t] + b_c)$$

Cell State

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t$$

Output Gate

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Hidden State

$$h_t = o_t tanh(C_t)$$

==========

Importance of Gates

- Forget gate removes unnecessary information.
- Input gate stores useful information.
- Output gate controls hidden state.

==========

Advantages of LSTM

- Solves vanishing gradient problem
- Learns long-term dependencies
- Better memory retention

==========

Applications

- NLP
- Speech recognition
- Translation
- Time-series forecasting

==========

Q9(a) Bahdanau Attention Mechanism

Need for Attention

Traditional encoder-decoder models compress all information into a single vector.

For long sentences this causes:

- Information loss
- Poor translation quality
- Weak context understanding

Attention solves this problem.

==========

Bahdanau Attention Steps

Step 1: Alignment Score

Compute similarity between decoder hidden state and encoder outputs.

Step 2: Softmax

Convert scores into probabilities.

Step 3: Attention Weights

Important words receive higher weights.

Step 4: Context Vector

Weighted sum of encoder outputs.

Step 5: Output Generation

Context vector combined with decoder state.

==========

Advantages

- Handles long sequences
- Improves machine translation
- Better context understanding
- Focuses on important words

==========

Applications

- Transformers
- Translation
- Speech recognition
- Image captioning

==========

Conclusion

Bahdanau attention significantly improves sequence learning by dynamically focusing on important information.

==========

Q9(b) YOLO Object Detection

Definition

YOLO (You Only Look Once) is a single-stage object detector.

It predicts:

- Bounding boxes
- Class labels
- Confidence scores

in a single forward pass.

==========

Difference Between YOLO and Two-Stage Detectors

YOLO	Two-Stage Detectors
Single pass	Region proposal + classification
Very fast	Slower
Real-time detection	Higher accuracy
Unified architecture	Multi-stage architecture

==========

Working of YOLO

Step 1: Divide Image into Grid

Image split into cells.

Step 2: Bounding Box Prediction

Each cell predicts boxes.

Step 3: Confidence Score

Measures object probability.

Step 4: Class Prediction

Predict object category.

Step 5: Non-Max Suppression

Remove duplicate boxes.

==========

Advantages

- Real-time performance
- Fast inference
- End-to-end training
- Detects multiple objects

==========

Applications

- Autonomous vehicles
- Surveillance
- Face detection
- Medical imaging

==========

Conclusion

YOLO achieves fast object detection by predicting all outputs simultaneously in a single neural network pass.

==========

Q10(a) VAE Problems and Solutions

Problem in VAE

VAEs often generate:

- Blurry images
- Low-quality samples
- Weak latent representations

==========

Causes

- Strong KL divergence
- Weak decoder
- Over-regularization

==========

Methods to Improve VAE

1. β-VAE

Controls KL divergence balance.

---

2. Stronger Decoder

Use deeper CNN decoder.

---

3. Perceptual Loss

Improves image sharpness.

---

4. Combine VAE with GAN

Produces realistic images.

==========

Conclusion

Advanced training methods improve VAE image quality significantly.

==========

Q10(b) GAN Face Generation Problem

Problem Statement

Need realistic anonymous faces with diverse facial expressions.

==========

Steps to Improve GAN

1. Use Large Diverse Dataset

Dataset must include:

- Multiple identities
- Different expressions
- Different lighting conditions

---

2. Data Augmentation

Apply:

- Rotation
- Flipping
- Cropping
- Noise

---

3. Use Wasserstein GAN

Improves training stability.

---

4. Batch Normalization

Improves convergence.

---

5. Conditional GAN

Generate specific facial expressions.

---

6. Feature Matching

Avoids mode collapse.

---

7. Balanced Generator and Discriminator Training

Prevents unstable learning.

==========

Advantages of GAN

- Highly realistic image generation
- High-quality face synthesis
- Diverse image generation

==========

Applications

- Deepfake generation
- Data augmentation
- Face anonymization
- Art generation

==========

Conclusion

Using advanced GAN architectures and stable training techniques ensures realistic and diverse face generation.

==========

END OF FULLY SOLVED FAT ANSWERS